

AI SAFETY ENGINEERING SERIES

Content Moderation for AI Safety

Protecting Online Communities with Azure AI Content Safety

A Comprehensive Guide for Aspiring Data Scientists & GenAI Engineers

This chapter is part of an ongoing series focused on building **end-to-end, production-ready GenAI and Data Science applications**. In this chapter, we explore one of the most critical components of any AI-powered product: **content moderation**. We will use **Microsoft Azure AI Content Safety** — a powerful, enterprise-grade service — to understand both the theory behind AI content moderation and the hands-on implementation steps.

What You Will Learn: The fundamentals of AI-powered content moderation and why it matters

How Azure AI Content Safety works under the hood

Setting up Azure AI Content Safety from scratch on the Azure portal

Hands-on text moderation, image moderation, prompt shields, and blocklists

Using the Azure REST API and Python SDK to integrate content safety into your applications

SECTION 01 | Theory & Concepts

Section 1: Theory — Understanding AI Content Moderation

1.1 What Is Content Moderation?

Content moderation is the practice of monitoring, reviewing, and managing user-generated or AI-generated content in digital platforms to ensure it meets community standards, legal requirements, and safety guidelines. Without moderation, platforms risk becoming vectors for harmful content such as hate speech, graphic violence, self-harm material, or explicit sexual content.

Traditionally, content moderation was performed by human reviewers — a slow, expensive, and psychologically taxing process. With the rise of large language models (LLMs) and the explosion of AI-generated content, automated AI-powered moderation has become not just useful but essential.

Real-World Context: In 2023, generative AI tools enabled the creation of millions of pieces of synthetic content daily.

Platforms like social networks, forums, and e-commerce sites receive billions of messages per day — volumes impossible for human moderators alone.

Regulatory frameworks like the EU AI Act and the Online Safety Act (UK) are increasingly mandating automated safety controls on AI systems.

1.2 Why Content Safety Is a First-Class Concern in GenAI

When you build GenAI applications — whether chatbots, content generators, or AI assistants — you face a unique dual-sided challenge:

- **User-generated inputs (prompts):** Malicious users may attempt to inject harmful instructions, extract copyrighted material, or manipulate AI models through adversarial prompts (a technique called prompt injection).
- **AI-generated outputs (responses):** Even well-intentioned models can produce content that is violent, sexually explicit, or factually dangerous — especially when fine-tuned on unfiltered data or manipulated via clever prompting.

Content safety, therefore, must operate at two levels: it must screen what goes IN and what comes OUT of your AI system.

1.3 Introducing Azure AI Content Safety

Azure AI Content Safety is a Microsoft Azure managed service that provides machine-learning-powered APIs to detect harmful content across text and images. It is part of the broader Azure AI services ecosystem (formerly known as Azure Cognitive Services).

The service is designed to be:

- **Multilingual:** Detects harmful content across more than 100 languages.
- **Multi-modal:** Works on both text (strings, documents, chat messages) and images (JPEG, PNG, GIF, BMP).
- **Configurable:** Developers control severity thresholds and can define custom blocked terms.
- **API-first:** Fully accessible via REST API and Azure SDKs, making it easy to integrate into any technology stack.

1.4 The Six Analysis Capabilities of Azure AI Content Safety

Azure AI Content Safety provides six core analytical capabilities. In this chapter, we focus on the first four:

Capability	Description / Use Case
Analyze Text	Scans text content for four harm categories: sexual content, violence, self-harm, and hate speech. Returns a severity score and a binary allow/block judgement for each category.
Analyze Images	Detects harmful visual content in images — including sexual material, violent imagery, and harassment-related visuals. Crucial for platforms that allow image uploads.
Prompt Shields	A specialized feature that protects against prompt injection attacks on large language models. It validates both user messages and system-level documents to detect adversarial manipulation.
Protected Material Text Detection	Checks whether AI-generated text contains excerpts that match known copyrighted material (e.g., song lyrics, book passages). Helps organisations avoid accidental copyright infringement.
Groundedness Detection	Determines whether an AI model's response is grounded in (i.e., faithful to) the source documents provided in a retrieval-augmented generation (RAG) pipeline. Prevents AI hallucinations from being served to users.
Custom Categories	Allows organisations to define their own harm taxonomy beyond the built-in categories. Useful for domain-specific moderation (e.g., financial fraud, medical misinformation).

1.5 Understanding Severity Levels and Thresholds

When Azure AI Content Safety analyzes content, it returns a severity score for each harm category. Understanding this scoring system is critical for building effective moderation pipelines.

Severity Score Scale

Content is annotated on a four-point scale:

Severity Level	Meaning
----------------	---------

Safe (0)	No harmful content detected. Content is appropriate for general audiences.
Low (2)	Mildly harmful or borderline content. May be acceptable in adult platforms with appropriate controls.
Medium (4)	Moderately harmful content. Requires review; likely inappropriate for most platforms.
High (6)	Severely harmful content. Should almost always be blocked regardless of platform.

The threshold you set determines the cut-off point for blocking. For example, if you set the threshold to Medium, content scored as Medium or High will be blocked, while Safe and Low content will be allowed through.

Design Consideration: Choosing the right threshold is a product decision, not just a technical one.

A children's education platform should set thresholds at Safe (block anything above 0).

A creative writing platform for adults may allow Low-severity content.

A medical information platform may need custom categories for content involving self-harm that differs from general harm.

1.6 The Four Built-in Harm Categories

Azure AI Content Safety monitors four built-in harm categories for text analysis:

- **Violence:** Content that depicts, glorifies, or incites physical violence against persons or groups. Includes graphic descriptions of injury, torture, or weapons use in a harmful context.
- **Sexual:** Content that contains explicit or implicit sexual material, including nudity, graphic sexual descriptions, or content that sexualises minors.
- **Self-harm:** Content that encourages, glorifies, or provides instructions for self-harm, suicide, or eating disorders.
- **Hate Speech (Hate):** Content that attacks, dehumanises, or incites discrimination against individuals or groups based on characteristics such as race, gender, religion, sexual orientation, or disability.

1.7 Blocklists: Custom Term Filtering

Beyond the ML-driven harm categories, Azure AI Content Safety supports custom blocklists — curated lists of specific words or phrases that you want to automatically flag or block, regardless of the overall severity score.

Blocklists are useful in scenarios such as:

- Blocking competitor brand names from appearing in AI-generated marketing content.
- Preventing specific slang or coded language used by extremist communities that the general model may not recognise.
- Enforcing domain-specific restrictions (e.g., a legal platform blocking terms that could constitute legal advice).

Important: A blocklist match causes an immediate block regardless of the ML severity score.

For example, even if the overall text is rated as Safe by the harm classifier, a blocklist hit will still result in a Blocked judgement.

Blocklist terms support wildcard matching and are case-insensitive by default.

1.8 Prompt Shields: Defending Against Prompt Injection

Prompt injection is one of the most serious security risks in GenAI systems. In a prompt injection attack, a malicious user crafts input that manipulates the AI model to ignore its system instructions and perform unintended actions — such as revealing confidential information, bypassing safety filters, or producing harmful content on command.

Prompt Shields in Azure AI Content Safety analyse both:

- **Direct attacks:** Where the user's own message contains instructions designed to override the system prompt.
- **Indirect attacks:** Where external content (e.g., a document passed to the AI as context) contains embedded malicious instructions intended to hijack the model's behaviour.

SECTION 02 | Practical Implementation & Code

Section 2: Practical — Setting Up and Using Azure AI Content Safety

2.1 Prerequisites and System Requirements

Before starting, ensure you have the following:

Requirement	Details
Operating System	macOS, Windows 10/11, or any Linux distribution
Azure Account	An active Microsoft Azure subscription (free tier available at portal.azure.com)
Python Version	Python 3.8 or higher recommended
Python Packages	azure-ai-contentssafety, azure-core, requests, python-dotenv
Azure Portal Access	Web browser with access to portal.azure.com
Code Editor	VS Code recommended; Jupyter Notebook for interactive exploration

2.2 Step-by-Step: Creating an Azure AI Content Safety Instance

This section walks you through the Azure portal configuration required before any code can be written.

Step 1: Navigate to the Azure Marketplace

1. Log in to portal.azure.com with your Azure account.
2. In the top search bar, type "Azure AI" and select Azure AI Content Safety from the dropdown suggestions.
3. On the Azure AI Content Safety marketplace page, click the Create button.

Step 2: Configure the Instance

Fill in the resource creation form with the following settings:

Field	Recommended Value
Subscription	Select your active Azure subscription
Resource Group	Create new or use an existing one (e.g., my-ai-rg)
Region	East US (recommended for availability; choose nearest for latency)
Name	Choose a unique instance name (e.g., my-content-safety)
Pricing Tier	Free F0 for development and testing; Standard S0 for production

Accept all defaults on the Network and Identity tabs, then click Review + Create, followed by Create.

Step 3: Configure IAM Permissions (Access Control)

Once the resource is deployed, you need to assign yourself the correct role to use the API:

4. Navigate to your newly created Content Safety resource.
5. In the left sidebar, click Access control (IAM).
6. Click + Add > Add role assignment.
7. Search for and select the role **Cognitive Services User**.
8. On the Members tab, set Assign access to to User, group, or service principal, then select your Azure user account.
9. Click Review + assign to complete the role assignment.

Why This Matters: Without the Cognitive Services User role, your API calls will return 403 Forbidden errors.

For production applications, it is best practice to assign this role to a Service Principal (application identity) rather than a personal user account.

Step 4: Access Content Safety Studio

From the resource overview page, click the **Content Safety Studio** link. This browser-based interface lets you test all moderation features interactively before writing a single line of code — an excellent way to calibrate thresholds and understand API behaviour.

2.3 Hands-On: Text Moderation in Content Safety Studio

The Content Safety Studio provides a no-code playground that is ideal for understanding the service before integrating it programmatically.

Using the Studio

10. In Content Safety Studio, select Moderate text content > Try it out.
11. Acknowledge that the demo uses the connected resource instance.
12. In the text input area, type or paste your test content.
13. On the right panel (Configure filters), set the threshold level for each harm category. The default is Medium for all four categories.
14. Click **Run test**. The results will appear under the View results tab.

Interpreting the Results

The results panel shows a table with the following columns:

- **Category:** The harm category (Hate, Violence, Sexual, Self-harm).
- **Severity Level:** The model's assessment (Safe, Low, Medium, High).
- **Threshold:** The threshold you configured (e.g., Medium).
- **Judgement:** The final decision: Allowed or Blocked.

If the content is blocked, a banner at the top will read: This content has been Blocked — Rejected by filter in [Category] category.

2.4 Hands-On: Using Blocklists

You can augment the automatic harm detection with a custom blocklist of forbidden terms.

15. In the Studio, navigate to the Use blocklist tab.
16. Create a new blocklist by clicking + New blocklist and giving it a name (e.g., violent_words).
17. Add terms to the blocklist (e.g., "butchered", "slaughter").
18. Return to the Moderate text content screen and enable your blocklist under the Use blocklist tab.
19. Run a test with content that contains your blocked terms.

Even if the harm classifier rates the content as Safe, a blocklist match will still trigger a Blocked — Rejected by blocklist detection result, with the matching terms shown in the Blocklist detection panel.

2.5 Hands-On: Image Content Moderation

The process for image moderation mirrors text moderation:

20. In Content Safety Studio, select Moderate image content.
21. Upload a test image or use one of the provided sample images.
22. Configure thresholds for the image harm categories (Sexual, Violence).
23. Click Run test to receive the moderation judgement.

Image Moderation Scope: Azure AI Content Safety analyses images for: sexually explicit or suggestive content, graphic depictions of violence, and harassment-related imagery.

Supported formats: JPEG, PNG, GIF, BMP, TIFF, WEBP.

Maximum image size: 4 MB. Images are analysed at the pixel level, not by OCR (text in images requires separate text extraction first).

2.6 Production Code: Python Integration with Azure AI Content Safety SDK

The following section presents a complete, annotated Python implementation using the official Azure AI Content Safety SDK. Each cell is explained in detail so you can understand not just what the code does, but why each decision was made.

Cell 1 — Install Required Packages

```
# Cell 1: Install the Azure AI Content Safety SDK and supporting packages
# Run this cell once in your environment (Jupyter, VS Code, or terminal)

!pip install azure-ai-contentssafety==1.0.0 # Official Azure Content Safety
SDK
!pip install azure-core                      # Core Azure authentication +
HTTP utilities
!pip install python-dotenv                  # Loads .env files for secure key
management
```

Explanation: azure-ai-contentssafety: The official Microsoft SDK that wraps the REST API calls, handles authentication, serialisation, and retry logic automatically.

azure-core: A foundational library for Azure SDKs that provides the AzureKeyCredential class used for API key authentication.

python-dotenv: A best-practice security tool. NEVER hard-code API keys in your source code. Store them in a .env file and load them at runtime.

Cell 2 — Import Libraries and Load Environment Variables

```
# Cell 2: Import all necessary libraries and securely load credentials

import os                                # Access environment
variables                                variables
from dotenv import load_dotenv           # Load .env file into
os.environ
from azure.ai.contentsafety import ContentSafetyClient
from azure.core.credentials import AzureKeyCredential
from azure.ai.contentsafety.models import (
    AnalyzeTextOptions,                  # Request model for text
    analysis                             analysis
    AnalyzeImageOptions,                 # Request model for image
    analysis                             analysis
    ImageData,                           # Wraps image bytes or URL
    TextCategory,                        # Enum: HATE, SELF_HARM,
    SEXUAL, VIOLENCE
    ImageCategory                        # Enum: HATE, SELF_HARM,
    SEXUAL, VIOLENCE
)

# Load environment variables from the .env file
# Your .env file should contain:
#   CONTENT_SAFETY_ENDPOINT=https://<your-
instance>.cognitiveservices.azure.com
#   CONTENT_SAFETY_KEY=<your-api-key>
load_dotenv()

ENDPOINT = os.getenv("CONTENT_SAFETY_ENDPOINT")
KEY       = os.getenv("CONTENT_SAFETY_KEY")

print(f"Endpoint configured: {ENDPOINT is not None}")
print(f"Key configured:      {KEY is not None}")
```

Explanation: AzureKeyCredential: Wraps your API key in a credentials object that the SDK uses to attach the Ocp-Apim-Subscription-Key header to every request.

TextCategory and ImageCategory enums: Using enums (HATE, SELF_HARM, SEXUAL, VIOLENCE) instead of raw strings prevents typos and makes code easier to read.

The .env pattern: Create a file named .env in your project root. Add it to .gitignore immediately. This prevents accidental credential exposure in version control.

Validation prints: Always verify that environment variables loaded correctly before making API calls. A None endpoint or key will result in cryptic SDK errors.

Cell 3 — Initialise the Content Safety Client

```
# Cell 3: Create and configure the Azure Content Safety client

# The ContentSafetyClient is the entry point for all API interactions.
# It manages connection pooling, authentication headers, and base URL routing.
client = ContentSafetyClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY)
)

print("ContentSafetyClient initialised successfully.")
print(f"Targeting endpoint: {ENDPOINT}")
```

Explanation: The client is instantiated once and reused across all API calls in your application — this is more efficient than creating a new client per request, as it reuses the underlying HTTP connection pool.

In production, this client initialisation typically happens at application startup (e.g., in a FastAPI lifespan event or a Flask app factory function).

AzureKeyCredential is the simplest authentication method. For production multi-tenant applications, consider using DefaultAzureCredential with Managed Identity for keyless authentication.

Cell 4 — Analyze Text for Harmful Content

```
# Cell 4: Perform text content analysis across all four harm categories

def analyze_text(text: str, threshold: int = 4) -> dict:
    """
    Analyse text for harmful content using Azure AI Content Safety.

    Args:
        text      : The text string to analyse.
        threshold : Severity score at or above which content is considered
        blocked.

                    Values: 0=Safe, 2=Low, 4=Medium, 6=High.
                    Default is 4 (Medium threshold).

    Returns:
        A dictionary with per-category severity scores and overall judgement.
    """
    # Build the request object
```

```
request = AnalyzeTextOptions(text=text)

# Make the API call
# analyze_text() sends a POST to /contentsafety/text:analyze
response = client.analyze_text(request)

results = {}
blocked = False

# The response contains a list of category analysis objects
# Each object has: category (enum), severity (0/2/4/6)
for item in response.categories_analysis:
    category_name = item.category.value # e.g., "Hate", "Violence"
    severity       = item.severity      # 0, 2, 4, or 6
    judgement      = "Blocked" if severity >= threshold else "Allowed"

    results[category_name] = {
        "severity" : severity,
        "judgement": judgement
    }

    if severity >= threshold:
        blocked = True

results["overall"] = "BLOCKED" if blocked else "ALLOWED"
return results

# --- Test 1: Safe content ---
safe_text = "The quick brown fox jumps over the lazy dog."
print("=== Test 1: Safe Content ===")
print(analyze_text(safe_text))

# --- Test 2: Potentially harmful content ---
test_text = "I want to hurt someone badly today."
print("\n=== Test 2: Harmful Content ===")
print(analyze_text(test_text))
```

Explanation: AnalyzeTextOptions: The request data model. At minimum it requires the text field. You can also pass categories (to only check specific harm types), blocklistNames (to apply custom blocklists), and outputType.

`response.categories_analysis`: A list of `CategoriesAnalysis` objects. Each has a `.category` attribute (`TextCategory` enum) and a `.severity` attribute (integer: 0, 2, 4, 6).

Threshold logic: The threshold comparison (`severity >= threshold`) is done client-side in this example. In production you may wish to configure this server-side using the Studio or directly in the request.

Why a function? Wrapping the API call in a reusable function with a clear signature is a production pattern. It makes unit testing easier and decouples the moderation logic from your application's business logic.

Cell 5 — Analyze an Image for Harmful Content

```
# Cell 5: Perform image content analysis

import base64

def analyze_image_from_file(image_path: str, threshold: int = 4) -> dict:
    """
    Analyse a local image file for harmful visual content.

    Args:
        image_path : Local file path to the image (JPEG, PNG, etc.)
        threshold  : Severity threshold for blocking (0, 2, 4, or 6).

    Returns:
        Dictionary with per-category severity scores and overall judgement.
    """
    # Read the image file and encode it as base64
    # The API accepts images as base64-encoded binary data
    with open(image_path, "rb") as image_file:
        image_bytes = image_file.read()
        image_b64 = base64.b64encode(image_bytes).decode("utf-8")

    # Wrap in an ImageData object (base64 encoded content)
    image_data = ImageData(content=image_bytes) # SDK accepts raw bytes

    # Build the request
    request = AnalyzeImageOptions(image=image_data)

    # Make the API call
    # analyse_image() sends a POST to /contentsafety/image:analyze
    response = client.analyze_image(request)
```

```
results = {}
blocked = False

for item in response.categories_analysis:
    category_name = item.category.value
    severity      = item.severity
    judgement     = "Blocked" if severity >= threshold else "Allowed"
    results[category_name] = {"severity": severity, "judgement":
judgement}
    if severity >= threshold:
        blocked = True

results["overall"] = "BLOCKED" if blocked else "ALLOWED"
return results

# Usage example:
# result = analyze_image_from_file("test_image.jpg")
# print(result)
print("Image analysis function defined. Call analyze_image_from_file(path) to
use.")
```

Explanation: ImageData: The SDK model for image input. It accepts content (raw bytes) or url (a publicly accessible image URL). Using bytes is preferred for private images.

base64 encoding is the standard format for binary data transmission over HTTP/JSON APIs. The SDK handles the encoding internally when you pass raw bytes.

Image restrictions to remember: Maximum file size is 4 MB. Very small images (under 50x50 pixels) or highly abstract images may yield lower-confidence results.

Reuse of logic: Notice the result processing logic is identical to the text version. In a real project, you would refactor this into a shared helper function to follow the DRY (Don't Repeat Yourself) principle.

Cell 6 — Working with Blocklists via the API

```
# Cell 6: Create and use a custom blocklist programmatically

from azure.ai.contentsafety import BlocklistClient
from azure.ai.contentsafety.models import (
    AddOrUpdateTextBlocklistItemsOptions,
    TextBlocklistItem
)
```

```
# The BlocklistClient manages blocklist CRUD operations
blocklist_client = BlocklistClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY)
)

BLOCKLIST_NAME = "violent_language_list"

# Step 1: Create or update the blocklist
# create_or_update_text_blocklist is idempotent - safe to call repeatedly
blocklist_client.create_or_update_text_blocklist(
    blocklist_name=BLOCKLIST_NAME,
    options={"description": "Terms flagged for violent connotation"}
)
print(f"Blocklist '{BLOCKLIST_NAME}' created/updated.")

# Step 2: Add terms to the blocklist
# Each term is a TextBlocklistItem with a text field
items_to_add = AddOrUpdateTextBlocklistItemsOptions(
    blocklist_items=[
        TextBlocklistItem(text="butchered"),
        TextBlocklistItem(text="slaughter")
    ]
)
blocklist_client.add_or_update_blocklist_items(
    blocklist_name=BLOCKLIST_NAME,
    options=items_to_add
)
print("Blocklist terms added: 'butchered', 'slaughter'")

# Step 3: Analyse text WITH the blocklist applied
test_text_with_blocklist = "The chef slaughter the vegetables expertly."

request_with_blocklist = AnalyzeTextOptions(
    text=test_text_with_blocklist,
    blocklist_names=[BLOCKLIST_NAME], # <-- attach your blocklist here
    halt_on_blocklist_hit=True        # Stop processing immediately on a hit
)

response = client.analyze_text(request_with_blocklist)
print(f"\nText: {test_text_with_blocklist}")
print(f"Blocklist hit: {response.blocklists_match}")
print("Categories analysis:")
```

```
for item in response.categories_analysis:
    print(f"    {item.category.value}: severity={item.severity}")
```

Explanation: BlocklistClient: A separate client specifically for managing blocklists. This separation of concerns reflects Microsoft's API design pattern.

create_or_update_text_blocklist: Idempotent operation. If the blocklist doesn't exist, it is created. If it does, it is updated. This makes it safe to call in deployment scripts.

blocklist_names parameter in AnalyzeTextOptions: This is the key link. You must explicitly pass the blocklist name when making an analysis request. Blocklists are NOT applied automatically.

halt_on_blocklist_hit=True: A performance optimisation. When True, the API stops processing as soon as a blocklist term is found, reducing latency. Set to False if you need complete analysis results even when a term is found.

response.blocklists_match: A list of TextBlocklistMatchResult objects showing which terms were matched and which blocklist they came from. Useful for logging and audit trails.

Cell 7 — Building a Production Moderation Pipeline

```
# Cell 7: A production-ready content moderation wrapper class

from dataclasses import dataclass
from typing import Optional

@dataclass
class ModerationResult:
    """Structured result object for content moderation decisions."""
    is_safe      : bool
    overall      : str          # "ALLOWED" or "BLOCKED"
    categories   : dict         # {category_name: {severity, judgement}}
    blocklist_hits: list        # List of matched blocklist terms
    reason       : Optional[str] # Human-readable reason if blocked

class ContentModerationPipeline:
    """
    Production-ready wrapper around Azure AI Content Safety.

    Designed to be instantiated once and reused across requests.
    Suitable for integration with FastAPI, Flask, Django, or any Python
    service.
    """
```



```
def __init__(self, endpoint: str, key: str,
              threshold: int = 4,
              blocklist_names: list = None):
    self.client = ContentSafetyClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(key))
    self.threshold = threshold
    self.blocklist_names = blocklist_names or []

def moderate(self, text: str) -> ModerationResult:
    """
    Run text through the full moderation pipeline.
    Returns a structured ModerationResult with all details.
    """
    request = AnalyzeTextOptions(
        text=text,
        blocklist_names=self.blocklist_names,
        halt_on_blocklist_hit=False # Always get full analysis
    )

    try:
        response = self.client.analyze_text(request)
    except Exception as e:
        # In production: log the error, potentially fail-open or fail-
closed
        # depending on your risk model
        print(f"Content Safety API error: {e}")
        # Fail-closed: treat as blocked on API error (safer for high-risk
platforms)
        return ModerationResult(
            is_safe=False, overall="BLOCKED",
            categories={}, blocklist_hits=[],
            reason="Content safety check unavailable"
        )

    categories = {}
    blocked = False
    blocked_by = []

    for item in response.categories_analysis:
        sev = item.severity
        cat_name = item.category.value
```

```
        allowed = sev < self.threshold
        categories[cat_name] = {"severity": sev,
                                "judgement": "Allowed" if allowed else
                                "Blocked"}

        if not allowed:
            blocked = True
            blocked_by.append(f"{cat_name} (severity={sev})")

    # Check blocklist hits
    blocklist_hits = []
    if response.blocklists_match:
        for match in response.blocklists_match:
            blocklist_hits.append(match.blocklist_item_text)
        blocked = True

    reason = None
    if blocked:
        parts = []
        if blocked_by:
            parts.append("Harm categories: " + ", ".join(blocked_by))
        if blocklist_hits:
            parts.append("Blocklist terms: " + ", ".join(blocklist_hits))
        reason = "; ".join(parts)

    return ModerationResult(
        is_safe      = not blocked,
        overall      = "BLOCKED" if blocked else "ALLOWED",
        categories    = categories,
        blocklist_hits = blocklist_hits,
        reason        = reason
    )

# --- Usage ---
pipeline = ContentModerationPipeline(
    endpoint=ENDPOINT,
    key=KEY,
    threshold=4, # Block Medium and above
    blocklist_names=["violent_language_list"]
)

texts_to_test = [
    "Today I learned a new Python trick for data analysis!",
```

```

    "I want to cause harm to people around me.",
    "The butcher slaughtered the lamb for the feast."
]

for text in texts_to_test:
    result = pipeline.moderate(text)
    print(f"\nText      : {text[:60]}...")
    print(f"Verdict   : {result.overall}")
    if result.reason:
        print(f"Reason    : {result.reason}")

```

Explanation: @dataclass ModerationResult: Using Python dataclasses for structured return types is a production best practice. It makes results easy to serialise to JSON, log, and test compared to returning raw dictionaries.

ContentModerationPipeline class: Encapsulating the client, threshold, and blocklist configuration in a class follows the Dependency Injection pattern. In FastAPI, you would create this once at startup and inject it into route handlers.

Exception handling (fail-closed vs fail-open): The try/except block demonstrates the fail-closed pattern — when the API is unavailable, we block the content rather than allow it. This is the safer default for sensitive platforms. Lower-risk platforms might fail-open (allow content on API errors) to prioritise availability.

Structured logging readiness: The ModerationResult object with reason makes it trivial to log why content was blocked — critical for compliance, auditing, and debugging in production environments.

2.7 Quick Reference: Azure AI Content Safety at a Glance

Feature	Key Detail
Text harm categories	Hate, Violence, Sexual, Self-harm
Severity scale	0 (Safe), 2 (Low), 4 (Medium), 6 (High)
Threshold configuration	Set per-category in Studio or via request parameters
Blocklists	Custom term lists; match causes immediate block
Prompt Shields	Detects direct and indirect prompt injection attacks
Image analysis	JPEG/PNG/GIF/BMP/TIFF/WEBP; max 4 MB
Language support	100+ languages for text analysis

Free tier limit	Free F0 tier: 5,000 text records/month
SDK	azure-ai-contentssafety (Python, .NET, JavaScript, Java)
Authentication	API Key or Azure Managed Identity (recommended for prod)

Key Takeaways

- Content moderation is a non-negotiable component of any production GenAI application that handles user input or generates public-facing content.
- Azure AI Content Safety provides ML-powered detection across four harm categories (Hate, Violence, Sexual, Self-harm) for both text and images.
- Severity thresholds are configurable — always tune them to your platform’s specific audience and risk tolerance.
- Blocklists offer a deterministic safety layer on top of ML-based detection, essential for domain-specific or compliance-driven moderation.
- Prompt Shields protect your LLM applications from adversarial prompt injection attacks at the input level.
- The Python SDK enables seamless integration into any service or API; wrap it in a class for clean, testable production code.
- Always store API keys in environment variables, never in source code. Use Managed Identity for production deployments.

Coming Up Next

Groundedness Detection & RAG Safety | Azure AI Studio | Building a Safe Chatbot End-to-End